SIMATS ENGINEERING

ASSESSMENT TOOL 3: Mock Interview

COURSE NAME: COMPUTER VISION COURSE CODE: ITA05

COURSE OUTCOME COVERED

CO5: *Develop computer vision applications using OpenCV for performing recognition and analysis on images and videos. (BTL 6)*

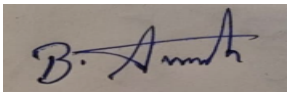
Assessment Tool Weightage: 10%

Code of Conduct:

I Avinash B (192511193) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Q2. *A video file is not loading properly in OpenCV. Analyze the possible causes and propose a structured debugging approach, explaining each step logically.*

Analysis: likely causes

- Incorrect file path (relative path resolved against the wrong working directory, or a simple typo).
- Missing or unsupported codec — OpenCV's VideoCapture relies on an underlying backend (typically FFmpeg); if the file's codec isn't supported by the installed OpenCV build, VideoCapture opens but produces no valid frames.
- A corrupted or incomplete video file.
- An incorrectly selected or unavailable capture backend for the platform (e.g., forcing a Windows-only backend on Linux).
- An OpenCV build (especially some pip-installed variants) compiled without full video I/O support.

Structured debugging approach

- Step 1 — Verify the path independently with os.path.exists(path) before touching OpenCV at all, ruling out a simple path/typo issue first.
- Step 2 — Check cap.isOpened() immediately after cv2.VideoCapture(path). If this is False, the problem is confirmed to be at the open/decode stage, not somewhere later in the processing pipeline.
- Step 3 — Open the same file in a standard media player to confirm the file itself isn't corrupted, independent of OpenCV.
- Step 4 — Inspect cv2.getBuildInformation() to confirm the installed OpenCV build actually includes FFmpeg/video support.
- Step 5 — Try explicitly specifying a backend, e.g., cv2.VideoCapture(path, cv2.CAP_FFMPEG), to rule out backend auto-selection choosing the wrong one.
- Step 6 — If the codec is confirmed unsupported, re-encode the file to a broadly compatible format (e.g., H.264 in an MP4 container) via ffmpeg and retest — this isolates whether the codec truly was the root cause.

This sequence deliberately eliminates one variable at a time — path, file integrity, library capability, backend selection, then codec — rather than guessing randomly, so the actual root cause is identified with confidence before any fix is applied.

Q4. *An image appears too dark when read. Analyze the issue and propose modifications to your processing pipeline, justifying how your approach improves visibility.*

Analysis

Before fixing anything, I'd first determine whether this is a genuine low-light capture or a display/scaling bug — for example, a 16-bit image being shown without proper normalization to the 8-bit display range can look almost black even though the actual captured data isn't that dark. I'd check image.min()/image.max() to confirm the real pixel-value range before deciding which problem I'm actually solving.

Pipeline modifications

- If it's a genuine low-light image: apply CLAHE (Contrast Limited Adaptive Histogram Equalization, cv2.createCLAHE) rather than plain global histogram equalization. CLAHE enhances contrast locally, tile by tile, which improves visibility in dark regions without disproportionately amplifying noise the way a single global transform tends to in already-dark images.

- For a simpler linear boost where appropriate: cv2.convertScaleAbs(image, alpha, beta) to adjust gain/brightness directly.
- If it's actually a scaling/display bug: use cv2.normalize() to rescale pixel values to the full 0–255 range before display, or ensure correct bit-depth conversion rather than naive truncation.

Justification

Diagnosing the true cause first avoids 'fixing' the wrong problem — boosting brightness on an image that's actually a scaling bug wouldn't address the real issue. CLAHE specifically is the right choice for genuinely dark images because it improves visibility while controlling noise amplification, which a naive brightness/contrast stretch does not.

Q5. *A video stream drops frames during processing. Analyze the problem and redesign your approach to ensure smooth playback, explaining trade-offs and optimization techniques.*

Analysis

Frame drops typically happen when per-frame processing (e.g., detection or inference) takes longer than the time budget the video's frame rate allows, so the capture buffer fills and frames get skipped — this is worsened when capture and processing run synchronously in a single thread, so a slow processing step blocks the next frame from even being read.

Redesign

- Decouple capture from processing using a producer-consumer pattern: a dedicated capture thread continuously reads frames into a bounded queue, while a separate processing thread consumes at whatever pace it can sustain — so a slow processing step no longer blocks frame capture itself.
- If processing genuinely can't keep up even with threading, apply frame skipping (process every Nth frame) to prioritize responsiveness over exhaustive analysis.
- Offload heavy computation (e.g., neural network inference) to GPU acceleration where available.
- Use a bounded queue with a drop-oldest policy so the system degrades gracefully — always working on the most recent frame — instead of falling progressively further behind.

Trade-offs and justification

Threading eliminates blocking-related drops but adds synchronization complexity. Frame skipping trades completeness for real-time responsiveness — acceptable for applications like motion-based surveillance, but not for use cases needing frame-accurate analysis. Together these changes target the actual bottleneck (processing speed vs. arrival rate) directly, rather than just enlarging a buffer, which would only delay drops rather than prevent them.

Q6. *You need to process only every 5th frame of a video. Design an efficient solution and justify your approach, considering performance and accuracy.*

Design

```
cap = cv2.VideoCapture(source)
frame_index = 0
N = 5

WHILE cap.isOpened():
```

```
    ret, frame = cap.read()
    IF NOT ret:
        BREAK
    IF frame_index % N == 0:
        ProcessFrame(frame)          // expensive step -- only runs on 1 in 5 frames
    frame_index += 1
```

Justification

I still call cap.read() on every frame, because most compressed video formats use inter-frame prediction, so sequential decoding is generally required to advance the stream correctly — but decoding is cheap relative to the actual processing step. The modulo check skips the expensive analysis (feature extraction, detection, etc.) on 4 out of every 5 frames, cutting that dominant cost by 80% while still examining an evenly spaced, representative 20% of the video rather than an arbitrary subset. For sources supporting reliable random-access seeking (e.g., cv2.CAP_PROP_POS_FRAMES) I could jump directly to target frames, but that's less reliable for arbitrary-frame accuracy in many compressed formats and adds seek latency — so for most real-time or streaming scenarios, sequential read with conditional processing is the simpler, more robust choice.

Q8. *You need to extract frames from a video and save only those with significant changes. Analyze the problem and design a method, explaining your decision criteria and approach.*

Design

```
cap = cv2.VideoCapture(source)
prev_frame = None
threshold = 30          // tunable sensitivity
frame_count = 0

FUNCTION FrameDifferenceScore(f1, f2):
    gray1 = cv2.cvtColor(f1, cv2.COLOR_BGR2GRAY)
    gray2 = cv2.cvtColor(f2, cv2.COLOR_BGR2GRAY)
    diff = cv2.absdiff(gray1, gray2)
    RETURN Mean(diff)

WHILE cap.isOpened():
    ret, frame = cap.read()
    IF NOT ret: BREAK
    IF prev_frame IS NOT None:
        score = FrameDifferenceScore(prev_frame, frame)
        IF score > threshold:
            cv2.imwrite(f"frame_{frame_count}.jpg", frame)
    prev_frame = frame
    frame_count += 1
```

Decision criteria and justification

I use frame-to-frame mean absolute pixel difference (after grayscale conversion, and optionally a light Gaussian blur to suppress sensor-noise sensitivity) as the 'significant change' metric — it's computationally cheap and directly measures real visual change while being robust to minor noise that would otherwise trigger on essentially static frames. If this proves too sensitive to lighting flicker, I'd move to a more perceptually meaningful metric like SSIM (structural similarity).

This directly targets the actual goal — capturing meaningful visual change — rather than saving frames at a fixed time interval, which could miss brief significant events during a 'quiet' period or waste storage on visually

redundant frames during a static one. The tunable threshold lets sensitivity be calibrated per application without adding significant per-frame computational cost.